Epoch — Technical Design Document

A local-first, universal machine learning training platform

Version: 0.1.0 (design draft)
Date: 2026-10-19
Status: Draft for implementation

Table of Contents

Table of Contents 2

1. Executive Summary 4

1.1 What Epoch is . . . 4

1.2 Who it serves . . . 4

1.3 The problem . . . 4

1.4 Why local-first . . . 4

1.5 Key differentiators . . . 4

2. Product Requirements 5

2.1 Functional requirements . . . 5

2.2 Non-functional requirements . . . 5

3. User Journeys 7

3.1 Beginner training a classification model . . . 7

3.2 Engineer training a regression model . . . 7

3.3 Computer vision researcher (planned, R3) . . . 7

3.4 Audio researcher building a speech classification system (planned, R3) . . . 7

3.5 Advanced user configuring a model manually . . . 7

3.6 Exporting and integrating a model . . . 7

4. System Architecture 8

4.1 Component overview . . . 8

4.2 Component responsibilities and boundaries . . . 8

4.3 Process model . . . 9

4.4 Training data flow . . . 9

4.5 Workspace layout on disk . . . 9

5. Universal Training Interface 10

5.1 Principle: shared orchestration, domain-specific data . . . 10

5.2 Contracts . . . 10

5.3 How heterogeneous domains coexist . . . 10

6. Dataset Intelligence 12

6.1 Capabilities and methods . . . 12

6.2 Why datasets need different handling . . . 12

7. AutoML and Model Selection 13

7.1 Pipeline . . . 13

7.2 Training modes . . . 13

7.3 Honesty statement . . . 13

8. Evaluation and Research Integrity 14

8.1 Protocol by task . . . 14
8.2 Rules enforced by the engine . . . 14
9. Compute and Hardware Management **15**
10. Desktop Application Design **16**
10.1 Technology choice . . . 16
10.2 Navigation and screen inventory . . . 16
10.3 Layout . . . 16
10.4 Representative wireframe . . . 16
10.5 States . . . 16
11. Website and Release Infrastructure **18**
12. Security and Privacy **19**
13. Data Models and Persistence **20**
14. Technology Stack **21**
15. Testing Strategy **22**
16. Implementation Roadmap **23**
17. Cost and Feasibility **24**
18. Limitations and Unresolved Decisions **25**
Glossary **26**
References **27**

1. Executive Summary

1.1 What Epoch is

Epoch is a downloadable, **local-first** desktop application that guides a user from a raw dataset to a trained, evaluated, exportable machine learning model without requiring the user to write training code. The user imports data, the engine inspects and validates it, the user states a learning objective, the engine proposes preprocessing and candidate models, trains them on the user's own hardware, and presents honest, leakage-aware evaluation results. The selected model is exported together with everything needed to reproduce inference.

Architecturally, Epoch is a **machine learning orchestration layer**. A thin, stable core coordinates datasets, tasks, experiments, resources and artifacts; domain-specific **providers** (tabular, vision, audio, text, time series) do the actual learning. The core never assumes that images, audio and tables share a representation.

1.2 Who it serves

Persona	Need	How Epoch helps
Beginner / analyst	Train a classifier from a spreadsheet without coding	Guided workflow, safe defaults, plain-language warnings
Engineer	Fast, reproducible baselines and exportable pipelines	Experiment tracking, deterministic splits, portable export
Researcher (vision/audio)	Rigorous evaluation protocol (group/speaker-independent)	Protocol-aware splitting, test-set protection, provenance
Advanced user	Manual control of every stage	Expert panels, config files, plugin interfaces

1.3 The problem

Most failures in applied ML are not caused by a lack of algorithms; they are caused by invalid evaluation (leakage, wrong split strategy, imbalanced metrics), irreproducible experiments, and glue code. Existing options are either code-first libraries, cloud AutoML services requiring data upload, or notebooks. Epoch targets the gap: a private, GUI-driven, evaluation-honest tool.

1.4 Why local-first

- **Privacy:** datasets (medical, voice, financial) never leave the machine by default.
- **Cost:** no subscription is required for ordinary local training.
- **Reproducibility:** the environment, seeds, library versions and data fingerprints are captured locally.
- **Honest limits:** local hardware bounds what can be trained; the product states this rather than hiding it.

1.5 Key differentiators

1. Evaluation protocol is a first-class, task-dependent object (group, temporal, stratified splits).
2. The test set is *protected*: the engine refuses to use it for selection and records any access.
3. Supported vs. planned functionality is explicit in the UI, docs and website.
4. Provider architecture lets heterogeneous domains coexist without a forced common representation.
5. Portable exports include preprocessing, schema, label map, metrics and a runnable example.

2. Product Requirements

Priority key: **R1** = initial release (v0.1–v0.3), **R2** = near-term, **R3** = long-term.

2.1 Functional requirements

ID	Area	Requirement	Release
F-01	Ingestion	Import CSV (delimiter/encoding sniffing, bounded preview)	R1
F-02	Ingestion	Import Excel (.xlsx), Parquet	R2
F-03	Ingestion	Import image folders (class-per-folder), audio folders + metadata CSV	R3
F-04	Inspection	Column type inference, missing-value counts, cardinality, duplicate rows	R1
F-05	Task	User selects target column; engine proposes classification vs regression and explains why	R1
F-06	Validation	Blocking errors vs. non-blocking warnings (leakage suspects, constant columns, ID-like columns)	R1
F-07	Splitting	Seeded stratified train/val/test; group-aware and temporal splits	R1 (stratified), R2 (group/temporal)
F-08	Preprocessing	Imputation, scaling, one-hot encoding fitted on training data only	R1
F-09	Training	Train baseline + candidate tabular models in a worker process with real progress	R1
F-10	Evaluation	Task-appropriate metrics on validation; single test-set evaluation for final model	R1
F-11	Experiments	Persist configs, metrics, versions and seeds in SQLite	R1
F-12	Registry	Store model artifacts with provenance and status	R1
F-13	Export	Export bundle: pipeline, schema, label map, config, metrics, requirements, example	R1
F-14	Monitoring	CPU/RAM utilization during training; GPU/VRAM where discoverable	R1 (CPU/RAM), R2 (GPU)
F-15	Control	Cancel training; pause/resume with checkpoints	R1 (cancel), R2 (pause/resume)
F-16	HPO	Budgeted random/Bayesian hyperparameter search with early stopping	R2
F-17	Neural	PyTorch backends for image/audio/text; fine-tuning pretrained models	R3
F-18	Plugins	Third-party providers via entry-point interface with trust prompts	R3
F-19	Time series	Forecasting and anomaly detection providers	R3

2.2 Non-functional requirements

ID	Category	Requirement
N-01	Privacy	No network access required for training; no telemetry without explicit opt-in
N-02	Reliability	A failed or cancelled run never corrupts the registry; partial runs are marked as such
N-03	Reproducibility	Same data fingerprint + config + seed + library versions → same split and (where the algorithm is deterministic) same metrics
N-04	Performance	UI thread never blocked by training or file parsing; preview loads ≤ 5 000 rows lazily
N-05	Accessibility	Keyboard navigable, sufficient contrast, scalable fonts, no colour-only encoding
N-06	Security	Untrusted files never executed; safe path handling; see Section 12

ID	Category	Requirement
N-07	Portability	Windows 10/11 x64 first; macOS and Linux later
N-08	Extensibility	New domain = new provider, no core change
N-09	Observability	Structured logs per run, diagnostic bundle export with redaction
N-10	Honesty	Each screen marks working vs. planned capabilities

3. User Journeys

3.1 Beginner training a classification model

1. Launches the app, chooses a workspace folder (default `Documents/Epoch`).
2. Drags in `customers.csv`. Engine shows 12 columns, types, missing counts.
3. Engine suggests `churned` as target (binary, low cardinality) and says *classification*.
4. Validation shows one warning: column `customer_id` is unique per row and will be excluded; another warns `refund_issued` may leak the target.
5. User accepts defaults. Engine builds a stratified 70/15/15 split with a fixed seed.
6. Engine trains a majority-class baseline, logistic regression and a gradient-boosted/random-forest model. Live progress is shown per candidate.
7. Results view highlights macro-F1 and balanced accuracy (not only accuracy), confusion matrix, and compares to the baseline.
8. User picks the best model, clicks *Export*, receives a folder with `model`, `schema.json`, `predict_example.py`.

3.2 Engineer training a regression model

Imports a CSV of sensor features, sets target `energy_kwh`, engine proposes regression, offers a temporal split because a datetime column exists. Engineer selects temporal split, adds Ridge and Random Forest candidates, inspects MAE/RMSE/R^2 and residual summaries, exports the bundle and integrates `predict()` into a service.

3.3 Computer vision researcher (planned, R3)

Imports a class-per-folder image dataset. Engine detects image sizes/corrupt files and near-duplicates (perceptual hash) across splits. Researcher selects fine-tuning of a pretrained backbone, sets a VRAM budget, trains with mixed precision, resumes from a checkpoint after interruption, and exports weights in a safe format (safetensors/ONNX).

3.4 Audio researcher building a speech classification system (planned, R3)

Imports audio plus a metadata CSV with `speaker_id`. Engine **forces speaker-independent splitting** (group split on `speaker_id`) and warns when a user attempts a random split. Feature extraction (log-mel) is cached to disk. Metrics: macro-F1, unweighted average recall, per-class confusion. Cross-validation is leave-speakers-out.

3.5 Advanced user configuring a model manually

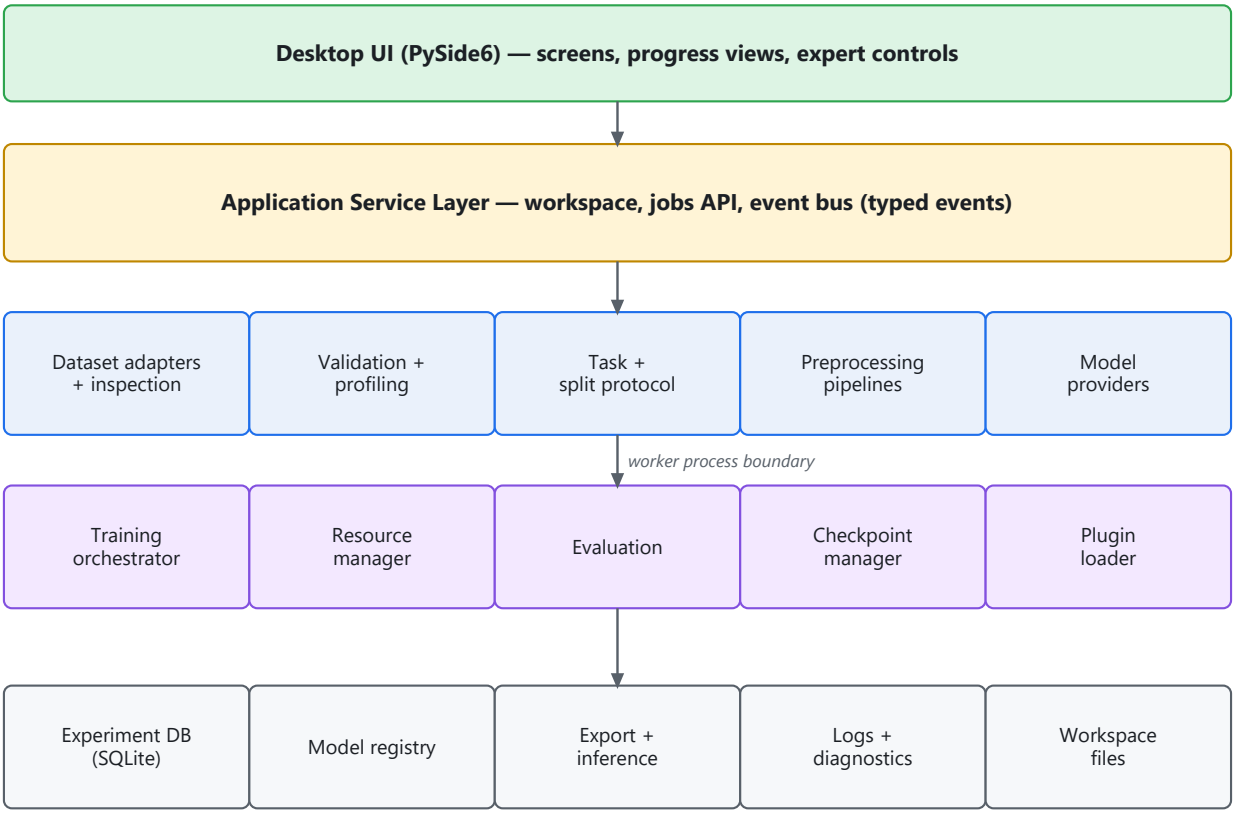
Opens *Expert* mode in the training configuration: edits a YAML/JSON experiment config, picks specific estimators and hyperparameters, disables automatic candidates, fixes seeds, selects scoring metric and CV scheme. The config round-trips to disk and can be re-run headlessly via the CLI.

3.6 Exporting and integrating a model

Opens the registry, selects a model, chooses *Export bundle*. Engine writes the artifact, input schema, preprocessing, label mapping, metrics report, `requirements.txt`, and an example script. A downstream app loads the bundle using the documented safe loader, validates incoming rows against `schema.json`, and calls `predict`.

4. System Architecture

4.1 Component overview



4.2 Component responsibilities and boundaries

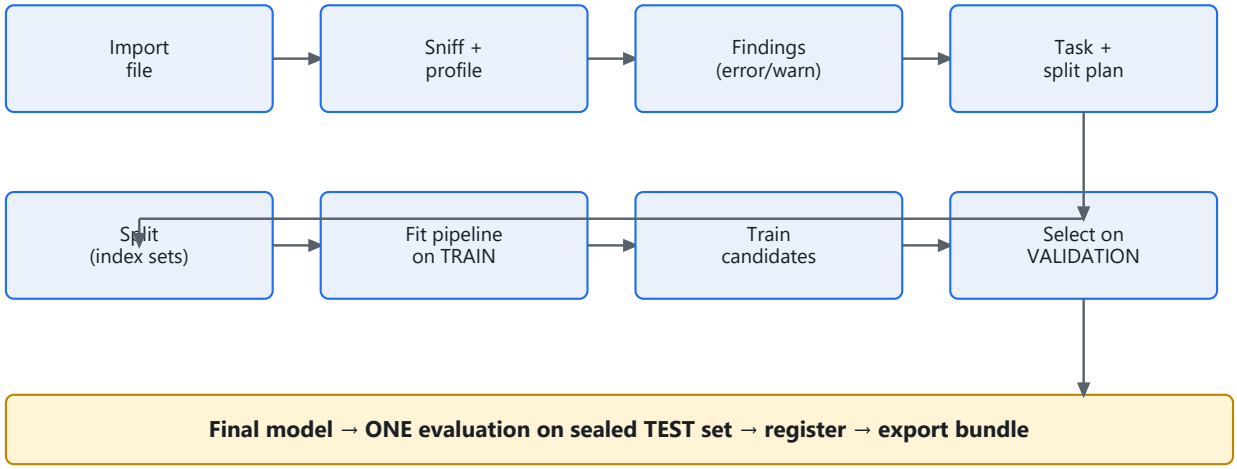
Component	Responsibility	Must NOT
Desktop UI	Presentation, user input, progress views	Run training or heavy parsing on the UI thread; import ML libraries in widgets
Dataset import & inspection	Safe file reading, profiling, preview	Execute content; load unbounded files into memory without limits
Dataset adapters	Domain-specific loading/representation (tabular, image, audio, text, TS)	Leak domain structure into the core
Validation & preprocessing	Findings (error/warn/info), fit-on-train transforms	Fit anything on validation/test data
Task identification	Propose task type, target, metric, split protocol	Silently override the user
Model-provider interface	Enumerate candidate models, build estimators, declare resource needs	Own persistence or UI
Training orchestration	Run plans, schedule candidates, stream events, handle cancel	Know model internals
Resource manager	Detect CPU/GPU, budget memory, choose devices	Guarantee success on unsupported hardware
Evaluation	Compute metrics, reports, enforce test-set protection	Be called with the test set during selection
Experiment tracking	SQLite records of runs, params, metrics, environment	Store large binaries (store paths + hashes)
Checkpointing & recovery	Periodic state saves, resume	Pickle untrusted data from other sources

Component	Responsibility	Must NOT
Model registry	Versioned model records, status, provenance	Delete artifacts without confirmation
Export & inference	Portable bundles, safe loaders, schema validation	Claim arbitrary model files are safe to load
Logging & diagnostics	Structured logs, redacted diagnostic bundles	Log raw data rows by default
Plugin interface	Entry-point provider discovery, trust prompt	Auto-run untrusted plugin code

4.3 Process model

The UI process owns the event loop. Training runs in a **separate worker process** (`multiprocessing`/subprocess) that communicates with the UI through a typed event stream (JSON lines over a queue). Benefits: crash isolation, true cancellation (terminate the process), GIL independence, and a clean boundary for resource limits.

4.4 Training data flow



4.5 Workspace layout on disk

```
<workspace>/
  epoch.sqlite        # experiment metadata
  datasets/<dataset_id>/       # source copy or reference + profile.json
  runs/<run_id>/               # config.json, events.jsonl, metrics.json, log.txt
  models/<model_id>/           # artifact, schema.json, report.json
  exports/<export_id>/         # portable bundles
```

5. Universal Training Interface

5.1 Principle: shared orchestration, domain-specific data

The core operates on **opaque handles and declared capabilities**, not on a universal tensor. A tabular dataset is a typed dataframe; an image dataset is a manifest of files plus labels; audio is a manifest plus a feature cache; text is a corpus plus tokenizer. Each adapter exposes the minimum the core needs: identity, fingerprint, size, group/time metadata, and a way to produce **splits as index sets**. The orchestrator moves index sets and configurations; only providers touch raw data.

5.2 Contracts

All contracts are Python `Protocol` classes (structural typing), versioned via `api_version`.

```
class DatasetAdapter(Protocol):
    api_version: str
    domain: str                                # "tabular" | "image" | "audio" | "text" | "timeseries"
    def can_open(self, path: Path) -> Confidence: ...
    def open(self, path: Path, options: dict) -> DatasetHandle: ...
    def profile(self, h: DatasetHandle) -> DatasetProfile: ...      # bounded, no code execution
    def fingerprint(self, h: DatasetHandle) -> str: ...              # content hash

class TaskDefinition(TypedDict):
    task_type: str            # classification | regression | detection | forecasting ...
    target: str | None
    primary_metric: str
    split_protocol: SplitProtocol   # stratified | group | temporal | kfold ...
    constraints: dict               # time budget, memory budget, device

class PreprocessingPipeline(Protocol):
    def fit(self, train: DataView) -> "PreprocessingPipeline": ...  # train indices only
    def transform(self, view: DataView) -> Any: ...
    def describe(self) -> dict: ...                                  # serialisable spec

class ModelProvider(Protocol):
    name: str
    def supports(self, task: TaskDefinition, profile: DatasetProfile) -> Support: ...
    def candidates(self, task, profile, budget) -> list[CandidateSpec]: ...
    def build(self, spec: CandidateSpec) -> TrainableModel: ...

class TrainingBackend(Protocol):
    def fit(self, model, train: DataView, val: DataView | None,
            callbacks: Callbacks, cancel: CancelToken) -> FitResult: ...

class Evaluator(Protocol):
    def evaluate(self, model, view: DataView, task: TaskDefinition) -> EvaluationReport: ...

class CheckpointManager(Protocol):
    def save(self, state: TrainState, run_dir: Path) -> CheckpointRef: ...
    def load_latest(self, run_dir: Path) -> TrainState | None: ...

class Exporter(Protocol):
    format: str               # "etbundle-sklearn" | "onnx" | ...
    def export(self, model_record, dest: Path) -> ExportManifest: ...
    def verify(self, dest: Path) -> list[Finding]: ...
```

5.3 How heterogeneous domains coexist

- **Capability negotiation:** `ModelProvider.supports()` returns `Supported | Partial | Unsupported(reason)`; the UI shows only what works and explains what does not.
- **Index-set splitting:** splitters return integer index arrays; they need group/time columns but never pixel or waveform data.
- **Per-domain DataView:** each adapter defines its `DataView` (dataframe slice, file list, spectrogram cache). The core passes it through opaquely.

- **Domain-specific preprocessing:** a log-mel extractor and a one-hot encoder both satisfy `PreprocessingPipeline`.
- **Per-domain evaluators:** mAP for detection and macro-F1 for classification are both `EvaluationReport` objects with a metrics map and artifacts.

6. Dataset Intelligence

6.1 Capabilities and methods

Capability	Method	User interaction
File type identification	Extension plus content sniffing (magic bytes, delimiter sniffing, encoding detection). Extension alone is never trusted.	Show detected type; allow override
Structure inspection	Row/column counts, dtypes, parse errors, ragged rows	Preview table
Missing/malformed data	Per-column null %, sentinel detection (`?`, `NA`, `-999`), mixed-type columns	Warning list
Candidate targets	Heuristics: name match (`label`, `target`, `y`, `class`), low-cardinality, last column, binary	Ranked suggestions; user confirms
Class imbalance	Class distribution, imbalance ratio; recommend macro-F1, class weights, stratification	Warning with explanation
Duplicates	Exact row hash; duplicates across splits flagged	Offer de-duplication before splitting
Target leakage	Single-feature predictive power probe on train folds (very high AUC/R^2 from one column), perfect-correlation, names resembling the target, post-event columns	Warning, never silent removal
Splitting strategy	Datetime column → propose temporal; group/ID column → propose group; else stratified (classification) or random (regression)	User confirms; unsafe choices warn
Ambiguity	When confidence < threshold, ask the user (e.g., "Is `zip` a category or a number?")	Inline question
Provenance	Keep original column names, source file path, SHA-256, import timestamp and original labels alongside any encoded versions	Shown in dataset details

6.2 Why datasets need different handling

A shuffled random split is acceptable for i.i.d. tabular rows but invalid when rows share a patient, a speaker, a session or a time window: near-identical samples land on both sides, inflating metrics. Imbalanced classes make accuracy misleading. Time-ordered data must not be shuffled. Image datasets often contain near-duplicates; audio datasets contain speaker identity. The profile is therefore not cosmetic: it drives the **split protocol, metric selection and warnings**.

7. AutoML and Model Selection

7.1 Pipeline

1. **Baselines first:** majority class / mean predictor, so every result is contextualised.
2. **Candidate generation:** the provider lists estimators suited to size, dimensionality and task (e.g., regularised linear model, random forest, gradient boosting).
3. **Preprocessing:** per-family (scaling for linear models; none for trees; ordinal/one-hot for categories), always inside a pipeline.
4. **Hyperparameter optimisation (R2):** random search then Bayesian (e.g., Optuna) over small, documented spaces, scored by cross-validation on training data only.
5. **Validation-based selection:** candidates compared on validation metrics (or CV mean ± std). Ties broken by simplicity and training cost.
6. **Resource budgets:** wall-clock and memory budgets per candidate; candidates exceeding budgets are stopped and recorded as such.
7. **Early stopping:** for iterative learners via validation loss patience.
8. **Final selection and one-shot test evaluation:** the chosen model is evaluated once on the test set; the result is stored and flagged as consumed.

7.2 Training modes

Mode	Description	Status
From scratch	Fit a new model on the user's data	R1 (tabular)
Fine-tuning	Adapt a pretrained model; requires licence/provenance tracking and often a GPU	R3
Inference with existing model	Load a model for prediction only; subject to safe-loading rules	R2

7.3 Honesty statement

Automated selection searches a finite, pre-declared space under a budget. It cannot guarantee an optimal model for every problem, and the UI reports the search space actually explored.

8. Evaluation and Research Integrity

8.1 Protocol by task

Situation	Required protocol	Metrics
i.i.d. classification	Stratified train/val/test	Accuracy, macro-F1, balanced accuracy, per-class P/R
Imbalanced classification	Stratified; report class weights used	Macro-F1, balanced accuracy, PR-AUC
Regression	Random or temporal split	MAE, RMSE, R^2, residual summary
Grouped data (patients, speakers)	**Group split**; groups never cross splits	Same as task; per-group variance reported
Time-dependent data	**Temporal split** (train < val < test in time)	Forecast metrics (MAE, MAPE with caveats)
Small datasets	Repeated/nested CV on training portion	Mean ± std
Audio (speaker-independent)	Leave-speakers-out CV	Macro-F1, UAR

8.2 Rules enforced by the engine

- Preprocessing is fit on the training fold only (enforced by pipelines).
- The test set is sealed: selection APIs do not accept it; evaluating on it writes an audit record.
- Splits are deterministic given `(dataset_fingerprint, seed, protocol)` and stored as index lists with a hash.
- Every report lists seed, library versions, split protocol and metric definitions.
- Confusion matrices accompany every classification report; accuracy is never shown alone when classes are imbalanced.
- Repeated testing on the test set triggers a visible warning ("test set already consulted N times").

9. Compute and Hardware Management

Topic	Design
CPU execution	Default path. `n_jobs` bounded by a user setting (default: physical cores − 1)
GPU discovery	Query via vendor APIs (NVML/`torch.cuda`) when available; absence is normal, not an error
Device selection	User picks device per run; default = best available meeting provider requirements
VRAM monitoring	Sampled periodically; warning near capacity; OOM caught and reported with suggested batch size
Memory-aware batch sizing	Probe forward/backward with doubling batch until a safety fraction (e.g., 80%) of free memory
Data loading/caching	Memory-mapped arrays, on-disk feature caches keyed by content hash and transform config
Checkpoint recovery	Atomic write (temp + rename); resume from last good checkpoint; corrupted checkpoint falls back to previous
Cancellation	Cooperative `CancelToken` first, process termination after grace period
Pause/resume	Requires checkpoint-capable providers; others show *Pause unavailable*
Out-of-core	Chunked reading for large CSV/Parquet; incremental learners (`partial_fit`) when supported; otherwise a clear "dataset too large" message
Unsupported hardware	Capability checks produce actionable messages (e.g., "this model requires CUDA; falling back to CPU is not supported")

Realistic scope: laptops comfortably handle tabular data up to millions of rows (algorithm-dependent) and small neural models. Fine-tuning large vision/audio/text models and large-scale hyperparameter searches generally require a discrete GPU with substantial VRAM; Epoch will state this rather than attempt it silently.

10. Desktop Application Design

10.1 Technology choice

PySide6 (Qt for Python). Justification: native-looking cross-platform desktop UI, mature table/chart/threading primitives, direct integration with the Python ML stack with no IPC bridge, LGPL licensing. Trade-offs: larger bundle size than a native shell, and fewer web-style UI niceties than Electron/Tauri; an Electron/Tauri front-end would require a Python sidecar with an HTTP/IPC bridge, increasing complexity for the first release.

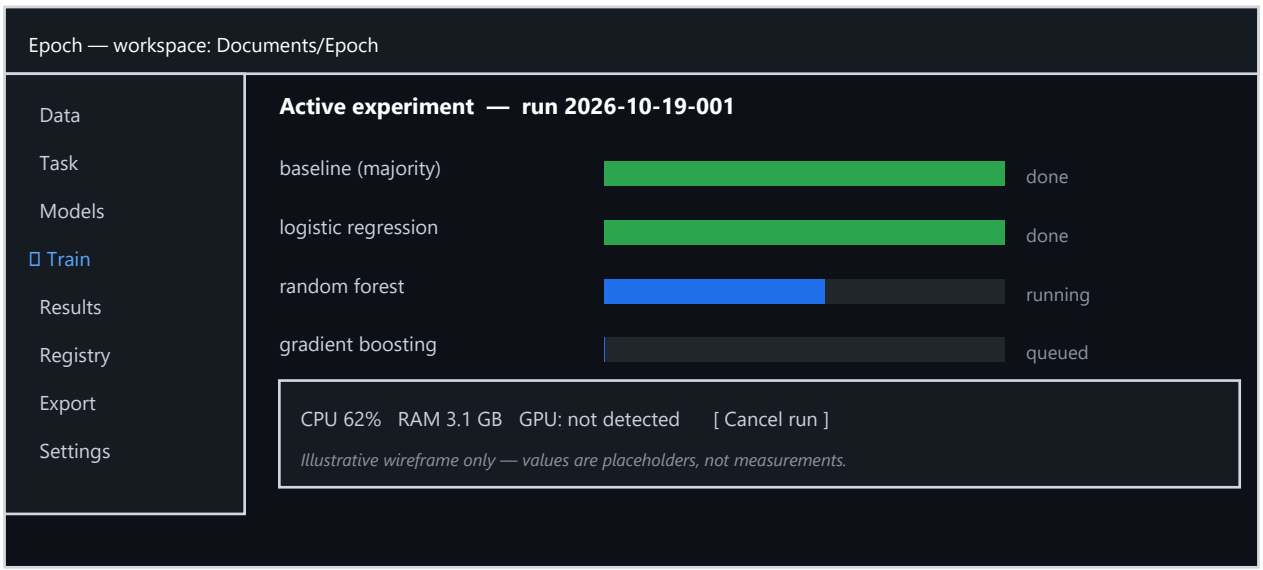
10.2 Navigation and screen inventory

#	Screen	Purpose	Status in v0.1
1	Welcome / workspace	Choose or create workspace, recent workspaces	Implemented
2	Dataset import	Pick file, delimiter/encoding, import	Implemented (CSV)
3	Dataset explorer	Preview rows, column types, missing, warnings	Implemented
4	Task configuration	Target, task type, split protocol	Implemented
5	Model selection	Candidate list with fit explanation	Implemented (tabular)
6	Training configuration	Seed, budgets, expert overrides	Partial
7	Active experiment monitor	Live progress, resource usage, cancel	Implemented
8	Results & comparison	Metrics, confusion matrix, model comparison	Implemented
9	Experiment history	Past runs, filter, reopen	Implemented
10	Model registry	Models, provenance, status	Implemented
11	Model export	Bundle export and verification	Implemented
12	Settings & diagnostics	Threads, paths, hardware, diagnostic bundle	Partial

10.3 Layout

A left navigation rail ordered as the workflow (Data → Task → Models → Train → Results → Registry → Export), a central work area, and a bottom status bar (workspace, active run, CPU/RAM). Future features appear **disabled with a "Planned" badge** and a tooltip stating the target release.

10.4 Representative wireframe



10.5 States

State	Behaviour
Empty	Each screen explains the next action and offers a single primary button
Loading	Non-blocking progress; long operations show determinate progress when possible
Error	Plain-language cause, technical details expandable, "copy diagnostics" action
Progress	Per-candidate rows with state (queued, running, done, failed, cancelled), elapsed time and live metric
Expert	Collapsible panels with raw config editing, seed, CV scheme, scoring metric

11. Website and Release Infrastructure

- **Static site** generated from templates and a versioned **release manifest** (`releases/releases.json`) validated against a JSON schema. No server required; deployable to any static host.
- **Pages:** Home, How It Works, Downloads, Documentation (installation, quick start, dataset preparation, training workflow, export, troubleshooting, system requirements, supported features, FAQ), Changelog & Roadmap, About & Contact.
- **Release data:** version, date, channel, status, artifacts (platform, architecture, package type, filename, URL, size, SHA-256, minimum OS, install notes), changelog, known issues.
- **Availability rule:** an artifact is *downloadable* only if the build step verifies that the file exists, matches its recorded SHA-256 and size, or (for remote artifacts) that a release job recorded a successful URL verification. Otherwise the page shows its state (*in development*, *planned*) and no download control.
- **Ordering:** semantic version descending within a channel; stable channel shown first.
- **Release validation:** CI computes checksums, runs an install smoke test (launch with `--selftest`), verifies the manifest schema and re-verifies URLs before publishing.
- **Documentation generation:** Markdown sources in `website/src/docs`, converted at build time.

12. Security and Privacy

Risk	Mitigation
Local dataset privacy	No network by default; no telemetry unless opted in; diagnostics redacted
Untrusted files	Sniff content, enforce size/row limits, parse with pure data parsers only; never `eval` or open with executable formats
Archive extraction	Reject path traversal (`..`, absolute paths, symlinks), cap total size and file count (zip bombs), extract into a fresh directory
Unsafe model serialisation	Pickle/joblib can execute code on load. Epoch's own bundles are loaded only after checksum verification against the bundle manifest and with a clear warning that **bundles from untrusted sources must not be loaded**. Preferred long-term formats: ONNX, safetensors, skops with trusted types
Dependency integrity	Pinned versions, hash-pinned lock files, SBOM at release
Subprocess isolation	Training in a separate process with resource limits; no inherited credentials
Plugin trust	Plugins are arbitrary code. Disabled by default; explicit user trust prompt showing source and hash; documented as equivalent to installing software
User-supplied code	Expert mode accepts configuration, not executable code, in v0.x. Any future scripting runs only after an explicit warning
Update integrity	Signed installers (planned) and published SHA-256

13. Data Models and Persistence

SQLite (WAL mode) stores metadata; large artifacts live on disk with path and SHA-256.

```
CREATE TABLE datasets (
  id TEXT PRIMARY KEY, name TEXT, source_path TEXT, sha256 TEXT,
  domain TEXT, n_rows INTEGER, n_cols INTEGER, profile_json TEXT, imported_at TEXT);

CREATE TABLE experiments (
  id TEXT PRIMARY KEY, dataset_id TEXT REFERENCES datasets(id),
  task_json TEXT, split_json TEXT, seed INTEGER, status TEXT,
  env_json TEXT, created_at TEXT, finished_at TEXT);

CREATE TABLE candidates (
  id TEXT PRIMARY KEY, experiment_id TEXT REFERENCES experiments(id),
  provider TEXT, estimator TEXT, params_json TEXT, status TEXT,
  fit_seconds REAL, val_metrics_json TEXT, error TEXT);

CREATE TABLE models (
  id TEXT PRIMARY KEY, candidate_id TEXT REFERENCES candidates(id),
  artifact_path TEXT, artifact_sha256 TEXT, format TEXT,
  selected INTEGER, test_metrics_json TEXT, test_consumed_at TEXT, created_at TEXT);

CREATE TABLE exports (
  id TEXT PRIMARY KEY, model_id TEXT REFERENCES models(id),
  path TEXT, manifest_json TEXT, created_at TEXT);

CREATE TABLE log_events (
  id INTEGER PRIMARY KEY AUTOINCREMENT, experiment_id TEXT,
  ts TEXT, level TEXT, event TEXT, payload_json TEXT);
```

Release manifest (website) entry:

```
{ "version": "0.1.0", "channel": "stable", "published": "2026-..-..", "status": "released",
  "changelog": ["..."], "known_issues": ["..."],
  "artifacts": [{ "platform": "windows", "arch": "x64", "package": "zip",
    "name": "Epoch-0.1.0-win-x64.zip", "url": "downloads/...", "size": 0,
    "sha256": "...", "min_os": "Windows 10", "install_notes": "..." }] }
```

Evaluation results are stored as JSON documents containing metrics, per-class tables, confusion matrix, split hash and library versions.

14. Technology Stack

Layer	Choice	Rationale / trade-off
Desktop UI	PySide6	See 10.1
Training engine	Python 3.12, scikit-learn, pandas, NumPy	Mature, deterministic, CPU-friendly
Tabular boosting	scikit-learn `HistGradientBoosting*`; XGBoost optional	Avoids a hard dependency; XGBoost added when available
Neural (future)	PyTorch	De facto standard; large install, so optional extra
Experiment DB	SQLite	Zero administration, file-based, transactional
Website	Static HTML/CSS/JS generated by a Python build script	No Node toolchain needed; trivially hostable; reproducible
Docs	Markdown → HTML at build time	Single source
PDF	ReportLab	Reliable, scriptable, pure-Python install
Packaging	PyInstaller (one-folder) then zip; installer (Inno Setup/MSIX) later	Practical for Windows; large bundles are a known trade-off
CI/CD	GitHub Actions (planned; no repository yet)	Matrix builds, artifact upload, checksum job
Export formats	Pickle-free where possible: skops/ONNX later; joblib bundle with checksum in v0.1	Safety vs. effort trade-off, documented in Section 12

15. Testing Strategy

Level	Scope
Unit	Type inference, validators, splitters (determinism, stratification, group integrity), metrics, manifest logic
Integration	Import → profile → split → train → evaluate → register → export on synthetic data
Synthetic-data	Generated datasets with known structure: separable classes, pure noise (metrics ≈ chance), planted leakage column (must be flagged), grouped rows (groups must not cross splits)
End-to-end	Headless workflow via CLI; later GUI automation
Model evaluation	Baseline-beating checks, reproducibility (same seed → same split/metrics), no preprocessing fit on held-out data
Packaging	Frozen app starts, imports resolve, `--selftest` passes
Release smoke	Manifest schema validation, checksum match, artifact existence, link check on built site

16. Implementation Roadmap

Milestone	Scope	Acceptance criteria
M0	Design, website, release manifest	PDF verified; site builds; unavailable artifacts have no download control
M1	Tabular engine (CSV)	Synthetic E2E test passes; leakage and group tests pass
M2	Desktop vertical slice	User completes import → export in the GUI on a real CSV
M3	Packaging	Frozen build launches and passes `--selftest`; checksum published
M4	AutoML (HPO), group/temporal splits, GPU monitoring	Budgeted search with CV; reproducibility tests
M5	Plugin API, Excel/Parquet	Trust prompt; schema contract tests
M6+	Vision, audio, text, time series providers (PyTorch)	Per-domain protocols and evaluators

First-release scope (v0.1): CSV, tabular classification and regression, stratified/random splits, 3–4 candidate models, SQLite tracking, CPU/RAM monitoring, bundle export. **Risks:** packaging size, cross-platform Qt quirks, pickle safety, user misinterpretation of metrics, scope creep into neural domains.

17. Cost and Feasibility

Item	Notes (no market statistics are asserted)
Development	Dominated by engineering time; the tabular slice is small, each additional domain is a substantial project
Hosting	Static site needs only a free/low-cost static host; downloads require file hosting or release assets
Compute	Ordinary use runs on the user's own machine; developer CI minutes for builds
Distribution	Windows code-signing certificates carry recurring cost; unsigned apps trigger SmartScreen warnings
Maintenance	Dependency updates, security patches, per-OS regressions, documentation upkeep

All cost figures must be gathered from current vendor pricing at planning time; none are estimated here.

18. Limitations and Unresolved Decisions

- Pickle-based export is not safe against malicious files; a safer format (ONNX/skops) decision is open.
- Leakage detection is heuristic and can miss or over-flag.
- Type inference can be wrong; users must be able to override it.
- Group/temporal split auto-detection is unreliable without user confirmation.
- Cross-platform packaging for macOS/Linux is not addressed in v0.1.
- Whether to bundle XGBoost (size, licensing review) is undecided.
- Plugin sandboxing is unsolved; trust prompt is the only planned control.
- Neural-network, vision, audio, text and time-series providers are **planned, not implemented**.
- Hyperparameter optimisation and pause/resume are planned (R2).
- GPU support depends on vendor libraries and is untested.

Glossary

Term	Meaning
Adapter	Component that opens a dataset of a given domain
Provider	Component that proposes and builds models for a domain
Leakage	Information in training features that would not be available at prediction time, or that leaks test data into training
Macro-F1	F1 averaged equally across classes
Balanced accuracy	Mean per-class recall
Group split	Split keeping all rows of a group (patient, speaker) on one side
Temporal split	Split ordered by time to avoid future information
Checkpoint	Saved training state allowing resume
Bundle	Exported directory with model, schema, metrics and examples
VRAM	GPU memory
UAR	Unweighted average recall

References

1. scikit-learn User Guide: Pipelines, cross-validation, model evaluation — https://scikit-learn.org/stable/user_guide.html
2. scikit-learn: Common pitfalls and recommended practices (data leakage) — https://scikit-learn.org/stable/common_pitfalls.html
3. Qt for Python (PySide6) documentation — https://doc.qt.io/qtforpython-6/
4. SQLite documentation — https://www.sqlite.org/docs.html
5. Python `pickle` documentation, security warning — https://docs.python.org/3/library/pickle.html
6. ReportLab User Guide — https://docs.reportlab.com/
7. PyInstaller manual — https://pyinstaller.org/en/stable/

References are cited as general documentation; no numerical claims in this document depend on them.